

Laya vs OpenJev as a local replacement for Jev

Can either open model do the decision work that jevgrep uses Jev for, on your own machine, for two jobs: finding the right code in a repository, and picking the right tools for an LLM?

Use case	■ Laya	■ OpenJev
a · Find the right code jevgrep: search a repo, return the relevant files and excerpts	✗ Not suitable Marks almost every file as relevant. jevgrep returned 76 of 113 files (391 KB) where Jev returned 6 files (6.9 KB).	✓ Suitable Separates relevant from irrelevant files well (AUC 0.92). Returned 5 files (9.4 KB) containing the code the real fix changes. Needs a GPU server: 3 h 12 min for one search on a Mac.
b · Pick the right tools shortlist tools from a fixed set of 20 before the LLM runs	! Usable, no better than keywords 87% right on the first pick, 96.4% in the top 3, in 0.04 s. Plain keyword search scored 90.8% and 96.2%.	✓ Suitable, best tested 98.5% right on the first pick, 100% in the top 3. About 4 s per request on a Mac; the model card reports about 80 ms per short decision on an H100.

0.59 Laya: how well it ranks the right files above others (0.5 = coin flip, 1.0 = perfect)	0.92 OpenJev: the same measure, on the same 214 files	85% smaller tool prompt when the LLM gets the top 3 tools instead of all 20
--	---	---

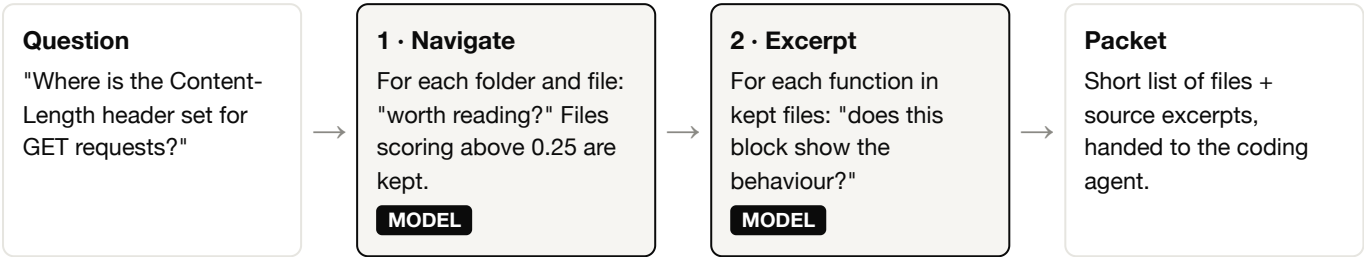
Bottom line. OpenJev is the only one of the two that does jevgrep's job. It needs a GPU server to be practical. Laya is fast and fine for picking among a handful of tools, but on this benchmark it did no better than keyword search, and it cannot judge source code.

No Jev API key was used. "Jev" figures come from a run recorded in the jevgrep repository. All tests ran on an Apple M3 Max (36 GB).

Where the decision model sits

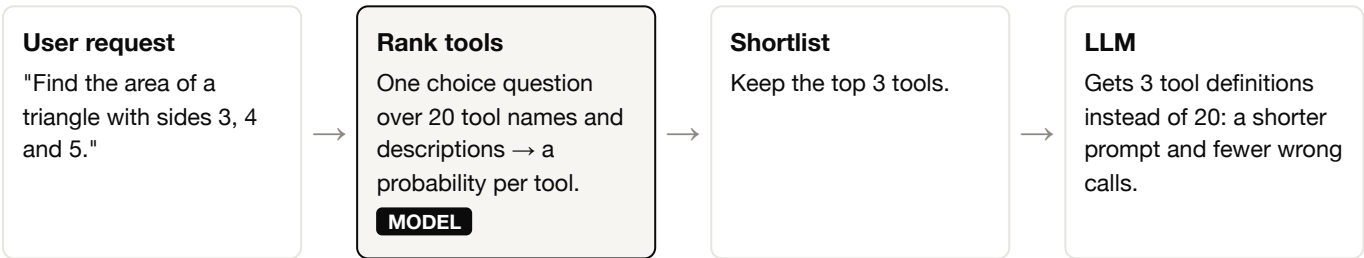
Jev, Laya and OpenJev are **decision models**: they do not write text. You give them a situation and a question, and they return a probability (yes/no) or a ranked choice. Both jobs below use that one skill.

a · Finding code with jevgrep



The whole value is in step 1: **a small packet saves the coding agent's tokens**. A model that says "yes" to everything turns the packet into a dump of the repository.

b · Picking tools for an LLM

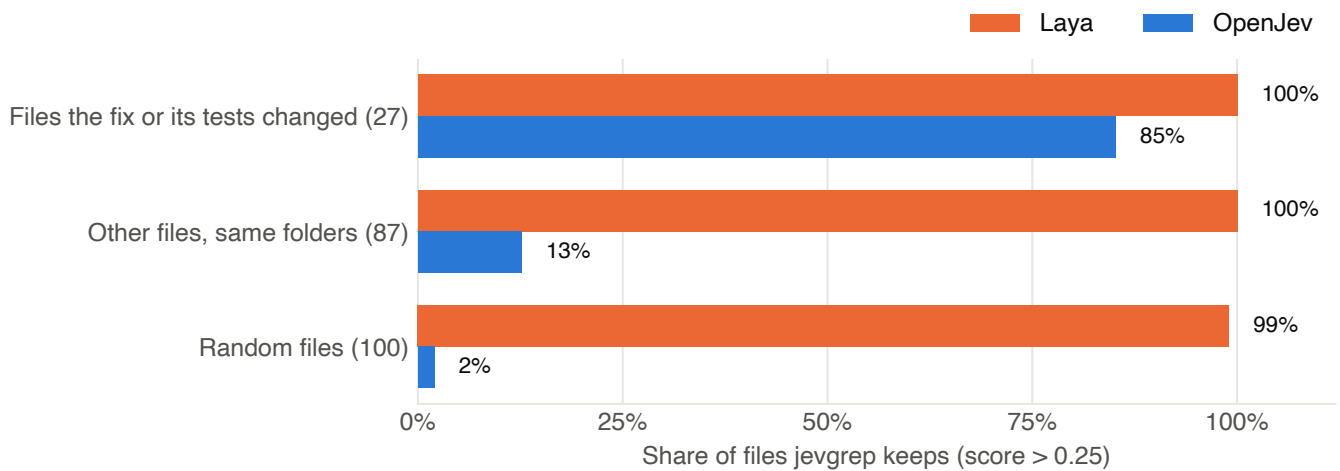


What was measured

Test	Data	Question answered
a1 · File relevance	10 real SWE-bench bugs; 27 files the fix or its tests changed, 187 other files	Does the model score the right files higher?
a2 · Full search	jevgrep end to end on the <code>psf/requests</code> bug (both models) and <code>pytest</code> (Laya)	What does the coding agent actually receive?
b · Tool choice	200 requests from the Berkeley Function Calling Leaderboard, 20 tools each	Is the right tool first, or in the top 3 or 5?

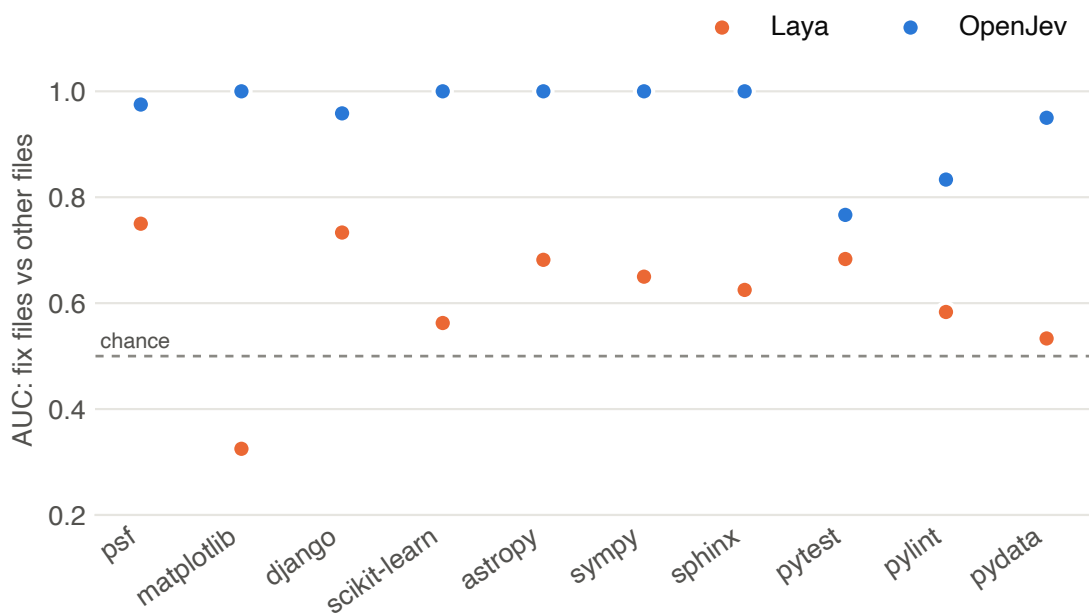
Laya keeps everything. OpenJev keeps the right files.

Each file was scored with jevgrep's own question: "does this file help investigate the requested behaviour?" jevgrep keeps a file when the score is above 0.25.



Laya kept **186 of the 187 unrelated files**, so it filters nothing. OpenJev kept 85% of the files the fix or its tests changed and only **13% of files in the same folders** and **2% of random files**. The 4 fix files it missed were secondary ones: two Django database backends, a pytest test file and a pylint constants file.

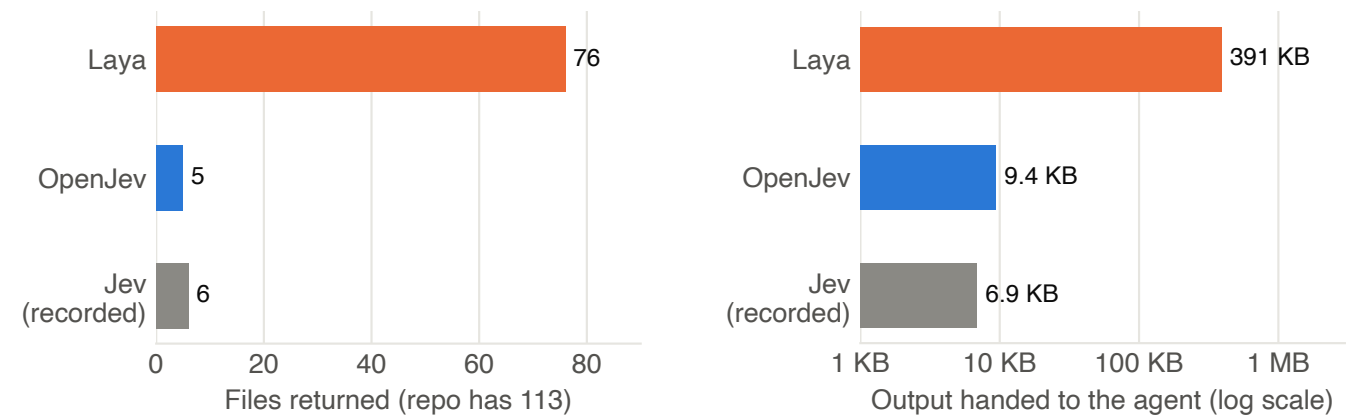
Is it consistent across projects?



AUC is the chance that a random fix file scores higher than a random other file. 0.5 is a coin flip. OpenJev scores 0.95 or higher on 8 of 10 projects and at least 0.77 on all of them. Laya ranges from 0.33 to 0.75, close to a coin flip (pooled against same-folder files: 0.59 for Laya, 0.92 for OpenJev).

What the coding agent actually receives

jevgrep run end to end on the `psf/requests` bug "GET requests always send a Content-Length header", with the query a coding agent really used. The Jev row is a run recorded in the jevgrep repository.



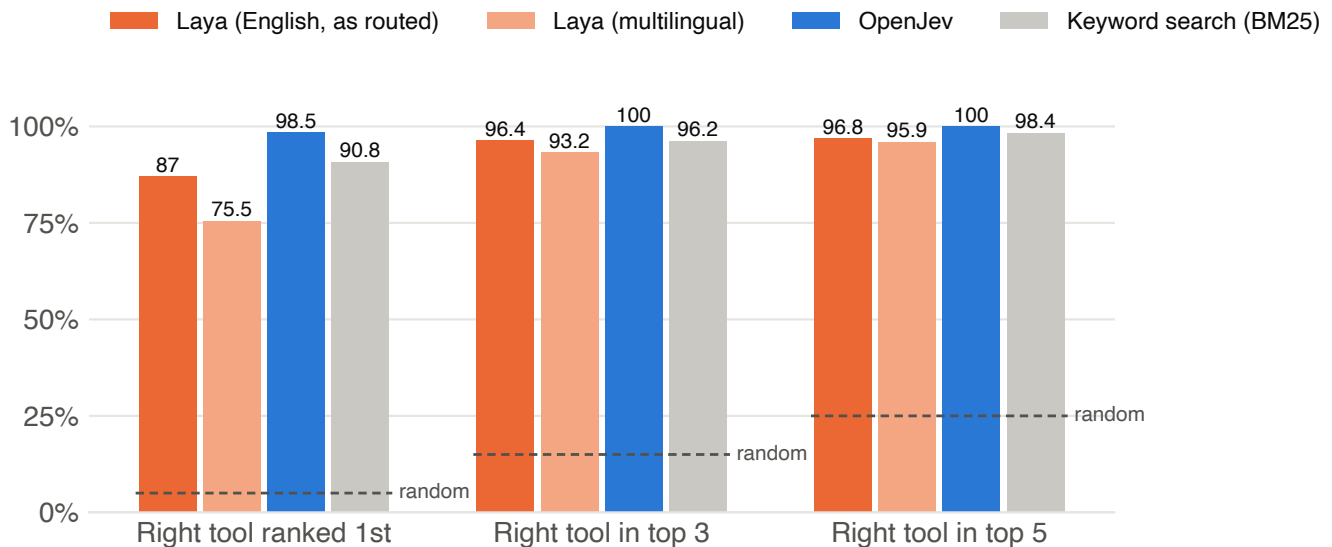
Model	Files	Output	Contains the code the fix changes?	Time (Mac)
Laya	76 of 113	391 KB	Yes, buried in two thirds of the repo	12 min
OpenJev	5	9.4 KB	Yes: <code>prepare_content_length</code> in <code>models.py</code>	3 h 12 min
Jev (recorded)	6	6.9 KB	Yes: <code>models.py</code> source blocks	–

On the larger `pytest` repository, Laya returned **410 of 449 files (3.8 MB)**, including 116 release notes. OpenJev was not run there: on this Mac each large repository would take many hours.

OpenJev's packet is the same kind of packet Jev produces: a handful of files centred on the right function. Laya's output is too large for an agent to use and would raise its cost instead of lowering it.

OpenJev almost always picks the right tool. Laya is no better than keyword search.

200 requests from the Berkeley Function Calling Leaderboard. Each request came with 20 tools: its own 2 to 4 candidates (often near-misses from the same domain, such as two triangle calculators) plus distractors from other requests, in shuffled order. Each model ranked the 20 tools once.



92.2% / 99%

Laya / OpenJev: picked the right tool among the near-miss candidates only (random: 38.4%)

10.1 → 1.5 KB

tool definitions sent to the LLM: all 20 tools vs the top 3

0.04 s / 4 s

Laya / OpenJev time per request on the Mac

How to read this

- **Top 3 is the number that matters for shortlisting.** If the right tool is in the top 3, the LLM can still pick it.
- **OpenJev** put the right tool first in 98.5% of requests and in the top 3 in all 200. Among the near-miss candidates alone it chose correctly 99% of the time. **Laya** put it first in 87% and in the top 3 in 96%, level with keyword search. Its multilingual checkpoint did worse (75.5% first).
- **Laya squeezes the tool list.** It fits all 20 options into 192 tokens (English checkpoint), so each tool keeps only about 8 tokens of its name and description. That limits it once tools get similar or numerous. Laya's own documentation recommends its shortlist mode above about 20 options.
- **This benchmark is easy for keywords.** The requests share many words with the tool descriptions, which is why BM25 does so well. Requests worded differently from the tool descriptions would favour the model that understands meaning.

What it takes to run them

	 Laya	 OpenJev
Model size	322M–421M parameters (3 checkpoints)	27B parameters; tested as the 4-bit Apple build (14 GB)
Reads up to	512–8,192 tokens, depending on checkpoint	16,384 tokens
Speed on this Mac	0.02–0.04 s per tool choice; about 2.5 s per <code>jevgrep</code> request	about 150 tokens/s: 4 s per tool choice, 25–40 s per file question
Speed on a GPU server	33 ms per question (model card, T4 GPU)	227 ms for a 1,100-token prompt (model card, one H100 with vLLM)
Licence	Apache 2.0	CC BY-NC 4.0: non-commercial use only without permission
Works with	Yes, via the patched <code>laya</code> provider	Yes, via the patched <code>openjev</code> provider

Recommendation

- **Code search (a):** use OpenJev on a GPU server: vLLM on one 80 GB NVIDIA GPU, or a 24 GB card with the GGUF build. The Mac is fine for testing but too slow for daily use. Check the licence before commercial use.
- **Tool picking (b):** use OpenJev when a GPU server is available: it put the right tool in the top 3 every time. Without a GPU, Laya or plain keyword search is a reasonable fast pre-filter for about 20 tools. Both put the right tool in the top 3 about 96% of the time, so send the LLM the top 3 to 5 tools, not just the first one.
- **Do not use Laya for code search.** It is trained on tickets and email, not source code.

Limits of these results

- Retrieval quality only. `jevgrep`'s own acceptance test, a coding agent solving the bug at lower cost, was not run.
- The full search was run on one bug for OpenJev and two bugs for Laya.
- OpenJev ran as the 4-bit build (about 0.6 points below the full model on its own benchmark).
- Tool test: one correct tool per request and 20 tools per request, on one public benchmark.
- "Jev" figures come from an earlier recorded run, not a paired live run.

Code, data and raw results: github.com/tabgab/jevgrep-laya, branch `laya-provider`, folder `evals/laya`.